demo-猴哥成语大冲关-后台开发功能文档

一、项目概述

本游戏是一款成语排序答题+每日冲关+月度身份晋升的休闲益智小游戏。核心玩法：玩家需要将打乱顺序的4个成语按正确顺序点击排列，在限定时间内连续答对5题通过一局，连续通过5局完成当日冲关。游戏包含时间双倍（广告增益）、跳过、复活（广告增益）等辅助功能，以及基于通关天数的月度身份晋升系统。

后端核心职责：

- 所有游戏结果的判定权（答对/答错、通关与否、冲关成功与否）
- 用户积分与身份的计算与发放
- 广告增益机会的管理与校验
- 风控体系的执行与标记
- 为前端和后台管理系统提供完整的 API（通过 Supabase Edge Functions 和直接数据库访问）

核心原则：前端只负责上报操作和展示结果，后端独立判定一切业务逻辑结果，绝不信任前端传来的任何结果数据。

二、系统架构

2.1 技术选型

类别	技术选型	说明
数据库	Supabase PostgreSQL	替代抖音云MySQL，支持JSON、RLS
API服务	Supabase Edge Functions	Deno运行时，替代抖音云函数
认证	Supabase Auth	匿名登录+手机号，替代openid体系
文件存储	Supabase Storage	替代抖音云存储
实时订阅	Supabase Realtime	可选，未来排行榜等功能
代码托管	GitHub	源码管理+CI/CD
部署	Supabase托管	数据库+Edge Functions一键部署

2.2 系统架构图（文字描述）

```
[Web前端 / 抖音小游戏前端]
  ↓ HTTPS (Supabase Client SDK)
[Supabase 平台]
  ├── Auth (匿名/手机号认证)
  ├── PostgreSQL (业务数据)
  ├── Edge Functions (业务逻辑)
  ├── Storage (文件存储)
  └── Realtime (实时订阅，可选)
[后台管理系统]
  ↓ Supabase Admin API / Dashboard
[Supabase PostgreSQL]
```

2.3 与原方案的核心差异

- 认证方式：openid → Supabase Auth UUID
- 数据库：MySQL → PostgreSQL（语法略有差异，JSONB替代JSON）
- API：云函数 → Edge Functions（Deno运行时，TypeScript）
- 管理：自建后台 → Supabase Dashboard + 自建管理页
- 部署：抖音云 → GitHub + Supabase CLI

2.4 核心安全设计

- Row Level Security (RLS)：所有表启用RLS，用户只能访问自己的数据
- JWT认证：所有请求自动携带Supabase Auth JWT
- Edge Functions中通过req.jwt获取用户身份，不信任前端传来的userId
- 服务端角色：Edge Functions使用service_role_key访问数据库，绕过RLS进行跨用户操作
- 频率限制：在Edge Functions中实现请求频率校验

三、Supabase 项目配置

3.1 项目创建与初始化

- 在 Supabase 控制台创建项目，获取 URL 和 API Keys

- 使用 Supabase CLI 进行本地开发：supabase init, supabase start
- 迁移管理：supabase migration new, supabase db push

3.2 环境变量

变量名	说明
SUPABASE_URL	项目URL
SUPABASE_ANON_KEY	前端安全密钥
SUPABASE_SERVICE_ROLE_KEY	服务端密钥（仅Edge Functions使用）
SUPABASE_DB_URL	数据库直连URL

3.3 Edge Functions 开发规范

- 每个Edge Function入口为index.ts，导出deredirect handler
- 使用Deno运行时，支持TypeScript
- 通过req.headers获取Authorization JWT
- 使用supabase-js的createClient进行数据库操作
- 错误处理统一返回ApiResponse格式

3.4 本地开发与调试

启动本地Supabase

supabase start

创建新Edge Function

supabase functions new session-start

本地运行Edge Function

supabase functions serve session-start --env-file .env.local

部署Edge Function

supabase functions deploy session-start

四、接口详细设计

4.1 接口通用规范

- 调用方式：supabase.functions.invoke('function-name', { body: {...} })
- 认证：自动携带JWT，Edge Functions中通过req.jwt验证
- 通用响应格式：ApiResponse<T> { code, message, data }

4.2 用户模块

4.2.1 初始化/登录 (user-init)

Edge Function: user-init

功能：用户首次进入或每次启动时调用

处理逻辑：

- 从JWT获取用户UUID
- 检查用户是否存在于users表
- 不存在：创建新用户（默认书生身份，userStatus=0）
- 存在：更新最后登录时间
- 检查风控状态（设备关联数、黑名单）
- 返回用户完整信息

响应数据示例：

```
{
  "code": 0,
  "data": {
    "userId": "uuid-xxx",
    "nickname": "玩家张三",
    "avatarUrl": "https://...",
    "userStatus": 0,
    "identityLevel": 3,
    "identityName": "举人",
    "totalPoints": 520,
    "currentMonthPoints": 120,
    "monthPassedDays": 8,
```

```
  "dailyStatus": {
    "todaySessions": 3,
    "todayPassed": false,
    "sessionResults": ["A", "A", "A", "F"],
    "canContinue": true,
    "maxSessionsReached": false
  },
  "adContributionScore": 7,
  "adRemaining": {
    "timeDouble": 3,
    "revive": 2
  }
 }
}
```


4.3 游戏核心模块

4.3.1 开始新一局 (session-start)

Edge Function: session-start

功能：玩家点击“开始游戏”后调用

处理逻辑：

- 查询今日对局数，若已达30局，返回maxSessionsReached=true
- 创建game_sessions记录（状态=进行中）
- 从题库中随机抽取5道不重复题目
- 计算本局初始倒计时：10-(N-1)秒（最低不低于3秒）
- 检查今日时间双倍次数是否还有剩余（<3次）且当前局数≤5局→canUseTimeDouble
- 返回局ID、题目列表、时间、跳过次数

响应数据示例：


```
{
 "code": 0,
 "data": {
  "sessionId": "sess_20260509_001",
```

"sessionSeq": 1,
"questions": [
{ "questionId": "q_001", "idioms": ["守株待兔", "兔死狐悲", "杯弓蛇影", "画蛇添足"] }
],
"initialTimeSeconds": 10,
"canUseTimeDouble": true,
"timeDoubleRemaining": 3,
"skipCountTotal": 3,
"skipCountRemaining": 3
}
}

4.3.2 提交答案 (session-answer)

Edge Function: session-answer

请求参数：{ sessionId, questionId, answer }

处理逻辑：

- 校验sessionId是否有效且属于当前用户
- 校验questionId是否为本局题目，是否已被提交过（防重复）
- 与正确答案对比，判定isCorrect
- 正确→答题计数+1，积分+1，更新game_sessions和points_log
- 错误→仅记录，不更新答题计数
- 检查是否答完5题→如果是，自动触发单局结束逻辑
- 返回判定结果

4.3.3 跳过题目 (session-skip)

Edge Function: session-skip

请求参数：{ sessionId, currentQuestionId }

处理逻辑：

- 检查本局剩余跳过次数是否>0
- 消耗一次跳过次数
- 从备用题库中随机抽取一道新题（不与已出题重复）
- 返回新题目数据

4.3.4 单局结束 (session-end)

Edge Function: session-end

请求参数：{ sessionId, isTimeout }

情况一：正常完成（isTimeout=false）

- 统计本局答对题数
- 答对5题→isSessionPassed=true，标记为A
- 答对<5题→isSessionPassed=false，标记为F
- 更新daily_challenge中的sessionResults数组
- 检查冲关连续性（是否存在连续5个A）

情况二：超时（isTimeout=true）

- 记录本局已答对题数
- 判定isSessionPassed=false
- 检查玩家是否还有复活机会（今日<2次）→canRevive
- 更新daily_challenge

冲关连续性检查算法：

- 遍历sessionResults数组
- 遇到“A”且前4局连续→冲关成功
- 遇到“R→A”视为“A”（保留连续性）
- 遇到“F”→连续性中断
- 遇到“R”→等待下一局结果

4.4 广告增益模块

4.4.1 使用时间双倍 (time-double-use)

Edge Function: time-double-use

请求参数：{ sessionId }

处理逻辑：

- 检查当日时间双倍使用次数<3
- 本局序号≤5
- 本局是否已用过时间双倍
- 广告回调验证通过后发放增益
- 本局初始时间×2

- 记录ad_opportunities.time_double_used + 1

4.4.2 复活确认 (revive-confirm)

Edge Function: revive-confirm

请求参数：{ failedSessionId }

处理逻辑：

- 校验广告回调已通过
- 生成新一局sessionId
- 在daily_challenge中将失败局标记从F改为R
- 返回新局数据（同session-start）
- 记录ad_opportunities.revive_used + 1

4.4.3 广告回调通知 (ad-callback)

Edge Function: ad-callback

说明：Demo阶段使用Mock，正式上线后由抖音平台广告服务端回调

请求参数：{ adType, userId, requestId, isEnded, ecpmValue }

处理逻辑：

- 根据requestId查找对应的广告请求记录
- 记录观看结果和eCPM值到ad_callback_logs
- isEnded=true→标记奖励可发放
- isEnded=false→奖励不发放
- 每日凌晨定时任务，基于ecpmValue和观看频次更新用户adContributionScore

4.5 用户数据模块

4.5.1 获取今日冲关进度 (daily-status)

方式：直接数据库查询（前端通过supabase.from('daily_challenge').select()）

RLS策略：用户只能查询自己的数据

五、数据库设计

5.1 PostgreSQL Schema

说明：将原MySQL DDL转换为PostgreSQL兼容语法，主要差异：

- AUTO_INCREMENT → SERIAL/BIGSERIAL
- JSON → JSONB（推荐，性能更好）
- TINYINT → SMALLINT
- DATETIME → TIMESTAMPTZ
- COMMENT → PostgreSQL COMMENT ON

5.2 用户表 (users)

```
CREATE TABLE users (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  openid VARCHAR(128) UNIQUE,
  unionid VARCHAR(128),
  nickname VARCHAR(64),
  avatar_url VARCHAR(512),
  mobile VARCHAR(20),
  region VARCHAR(64),
  user_status SMALLINT DEFAULT 0,
  identity_level INT DEFAULT 0,
  total_points BIGINT DEFAULT 0,
  current_month_points BIGINT DEFAULT 0,
  ad_contribution_score SMALLINT DEFAULT 1,
  last_login_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);
COMMENT ON COLUMN users.user_status IS '0正常 1异常 2非法';
COMMENT ON COLUMN users.identity_level IS '0书生 1童生 2秀才 3举人 4进士 5状元 6太傅 7皇帝';
COMMENT ON COLUMN users.ad_contribution_score IS '1-10';
CREATE INDEX idx_users_openid ON users(openid);
```

5.3 设备关联表 (user_devices) - 风控

```
CREATE TABLE user_devices (
  id BIGSERIAL PRIMARY KEY,
```

```
  user_id UUID NOT NULL REFERENCES users(id),
  device_id VARCHAR(256) NOT NULL,
  first_seen TIMESTAMPTZ DEFAULT now(),
  last_seen TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_user_devices_user ON user_devices(user_id);
CREATE INDEX idx_user_devices_device ON user_devices(device_id);
```

5.4 对局记录表 (game_sessions)

```
CREATE TABLE game_sessions (
  id BIGSERIAL PRIMARY KEY,
  session_id VARCHAR(64) NOT NULL UNIQUE,
  user_id UUID NOT NULL REFERENCES users(id),
  session_date DATE NOT NULL DEFAULT CURRENT_DATE,
  session_seq INT NOT NULL,
  questions_data JSONB,
  correct_count INT DEFAULT 0,
  points_earned INT DEFAULT 0,
  is_passed BOOLEAN DEFAULT false,
  skip_used INT DEFAULT 0,
  revive_used INT DEFAULT 0,
  time_double_used INT DEFAULT 0,
  total_time_seconds INT DEFAULT 0,
  status VARCHAR(20) DEFAULT 'playing',
  start_time TIMESTAMPTZ DEFAULT now(),
  end_time TIMESTAMPTZ
);
CREATE INDEX idx_game_sessions_user_date ON game_sessions(user_id, session_date);
```

5.5 每日冲关表 (daily_challenge)

```
CREATE TABLE daily_challenge (
  id BIGSERIAL PRIMARY KEY,
```

```
  user_id UUID NOT NULL REFERENCES users(id),
  challenge_date DATE NOT NULL DEFAULT CURRENT_DATE,
  session_results JSONB DEFAULT '[]'::jsonb,
  daily_passed BOOLEAN DEFAULT false,
  daily_points INT DEFAULT 0,
  total_sessions INT DEFAULT 0,
  time_double_used_today INT DEFAULT 0,
  revive_used_today INT DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(user_id, challenge_date)
);
```

5.6 广告机会管理表 (ad_opportunities)

```
CREATE TABLE ad_opportunities (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  opportunity_date DATE NOT NULL DEFAULT CURRENT_DATE,
  time_double_used INT DEFAULT 0,
  revive_used INT DEFAULT 0,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(user_id, opportunity_date)
);
```

5.7 广告回调日志表 (ad_callback_logs)

```
CREATE TABLE ad_callback_logs (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  ad_type VARCHAR(32) NOT NULL,
  session_id VARCHAR(64),
  request_id VARCHAR(128),
```

```
  is_ended BOOLEAN DEFAULT false,
  ecpm_value DECIMAL(10,4) DEFAULT 0,
  reward_granted BOOLEAN DEFAULT false,
  callback_time TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_ad_callback_logs_user_date ON ad_callback_logs(user_id,
DATE(created_at));
```

5.8 积分流水表 (points_log)

```
CREATE TABLE points_log (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  points_amount INT NOT NULL,
  points_source VARCHAR(32),
  related_session_id VARCHAR(64),
  description VARCHAR(256),
  created_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX idx_points_log_user ON points_log(user_id, created_at);
```

5.9 身份晋升记录表 (identity_log)

```
CREATE TABLE identity_log (
  id BIGSERIAL PRIMARY KEY,
  user_id UUID NOT NULL REFERENCES users(id),
  month_year VARCHAR(7) NOT NULL,
  old_level INT,
  new_level INT,
  month_passed_days INT DEFAULT 0,
  promotion_date TIMESTAMPTZ
);
CREATE INDEX idx_identity_log_user_month ON identity_log(user_id, month_year);
```

5.10 题库表 (questions)

```
CREATE TABLE questions (
  id BIGSERIAL PRIMARY KEY,
  question_id VARCHAR(64) NOT NULL UNIQUE,
  idioms_ordered JSONB NOT NULL,
  idioms_shuffled JSONB NOT NULL,
  difficulty INT DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT now()
);
COMMENT ON COLUMN questions.difficulty IS '1-5';
```

六、Row Level Security (RLS) 策略

说明：所有业务表必须启用RLS，确保用户只能访问自己的数据。Edge Functions使用service_role_key时可绕过RLS。

```
-- 用户表：用户只能读取自己的数据
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can read own data" ON users
  FOR SELECT USING (auth.uid() = id);
CREATE POLICY "Users can update own data" ON users
  FOR UPDATE USING (auth.uid() = id);

-- 对局记录：用户只能查看自己的对局
ALTER TABLE game_sessions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can read own sessions" ON game_sessions
  FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert own sessions" ON game_sessions
  FOR INSERT WITH CHECK (auth.uid() = user_id);

-- 每日冲关：用户只能查看自己的冲关数据
ALTER TABLE daily_challenge ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can read own challenge" ON daily_challenge
```

```
  FOR SELECT USING (auth.uid() = user_id);

-- 积分流水：用户只能查看自己的积分
ALTER TABLE points_log ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Users can read own points" ON points_log
  FOR SELECT USING (auth.uid() = user_id);

-- 题库：所有已认证用户可读
ALTER TABLE questions ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Authenticated users can read questions" ON questions
  FOR SELECT USING (auth.role() = 'authenticated');
```

七、后台管理系统

7.1 方案选择

Demo阶段直接使用Supabase Dashboard进行数据管理，无需自建管理后台。Supabase Dashboard提供表数据浏览、SQL编辑器、用户管理、日志查看等基础功能，满足MVP阶段需求。

7.2 后续扩展（二期）

- 自建管理页面（可使用Next.js + Supabase Admin API）
- 仪表盘：今日实时数据、广告收益、异常告警
- 用户管理：用户列表、风控标记、黑名单
- 题库管理：手动/批量添加题目
- 数据导出：CSV/报表
- 参数配置：热更新游戏参数

八、GitHub 部署与 CI/CD

8.1 仓库结构

```
chengyu-game-backend/
├── supabase/
│   ├── config.toml
│   ├── migrations/
```

```
│   │   ├── 001_create_users.sql
│   │   ├── 002_create_game_sessions.sql
│   │   └── ...
│   └── functions/
│       ├── user-init/
│       │   └── index.ts
│       ├── session-start/
│       │   └── index.ts
│       ├── session-answer/
│       │   └── index.ts
│       ├── session-skip/
│       │   └── index.ts
│       ├── session-end/
│       │   └── index.ts
│       ├── time-double-use/
│       │   └── index.ts
│       ├── revive-confirm/
│       │   └── index.ts
│       └── ad-callback/
│           └── index.ts
├── scripts/
│   ├── seed-questions.ts    # 题库初始化脚本
│   └── test-api.ts          # API测试脚本
├── .env.example
├── package.json
└── README.md
```

8.2 CI/CD 流程

- 推送到GitHub main分支自动触发Supabase CLI部署
- 数据库迁移自动执行
- Edge Functions自动部署

- 可使用GitHub Actions配置

8.3 本地开发流程

1. 克隆仓库

```
git clone https://github.com/cfqing202/chengyuguess2025.git
```

2. 安装依赖

```
npm install
```

3. 启动本地Supabase

```
supabase start
```

4. 执行数据库迁移

```
supabase db push
```

5. 初始化题库数据

```
npm run seed
```

6. 本地运行Edge Functions

```
supabase functions serve
```

7. 测试

```
npm test
```

九、开发实施建议

9.1 分阶段开发

第一阶段（MVP - 跑通Demo）：

- Supabase项目创建与配置
- 数据库Schema创建（迁移脚本）
- RLS策略配置

- Edge Function: user-init（用户初始化）
- Edge Function: session-start（开始新局）
- Edge Function: session-answer（提交答案）
- Edge Function: session-end（单局结束）
- 基础题库数据导入（至少100题）
- 每日冲关逻辑
- 积分系统
- 身份晋升计算

第二阶段（广告对接）：

- Edge Function: time-double-use
- Edge Function: revive-confirm
- Edge Function: ad-callback
- eCPM记录与贡献值计算
- 广告Mock→真实SDK切换

第三阶段（完善）：

- 后台管理系统
- 数据报表
- 礼品兑换系统
- 性能优化、缓存策略
- 抖音小游戏平台适配

9.2 安全清单

- 所有结果判定的代码不依赖前端传来的任何结果字段
- userId在Edge Functions中从JWT获取，不信任前端传来的userId
- 每局超过30次后的请求直接返回限制状态
- 敏感操作（发放积分、使用增益）必须记录日志
- service_role_key只在Edge Functions中使用，绝不暴露给前端
- RLS策略必须覆盖所有业务表
- 用户频率限制（答题接口每秒最多1次）

9.3 Demo阶段简化策略

- 广告系统使用Mock，前端和后端均Mock广告回调

- 不实现真实的eCPM计算，使用默认值
- 后台管理直接用Supabase Dashboard
- 不实现设备关联风控的复杂逻辑，仅做基础检查
- 题库通过SQL脚本批量导入